

synchronized 和 ReentrantLock 区别讲义

□ 考点定位

考**锁机制的对比辨析能力**——重点区分 JVM 内置锁与 JUC 显式锁在实现方式、功能边界、适用场景上的本质差异；**难度 □□□□**（需理解底层 + API + 工程权衡），**高频度 高频**（90% Java 中高级岗必问）。

□ 答题框架

`定义归属 → 获取释放方式 → 功能对比 → 性能演进 → 场景选型`

□ 核心知识点

1. **定义与归属**：

- `synchronized` 是 **JVM 关键字**，基于 **Monitor（监视器）** 实现，属于 **隐式锁**；
- `ReentrantLock` 是 **JDK 提供的类**（`java.util.concurrent.locks`），属于 **显式锁**，需手动 `lock()` / `unlock()`。

2. **获取与释放方式**：

- `synchronized`： **自动获取/释放**（进入同步块/方法时加锁，退出时自动释放，即使异常也保证释放）；
- `ReentrantLock`： **必须显式配对调用**，推荐 `try-finally` 保障释放：

```
```java\n    ReentrantLock lock = new ReentrantLock();\n    lock.lock(); // □ 必须调用\n    try {\n        // 临界区\n    } finally {\n        lock.unlock(); // □ 必须保障执行\n    }\n```
```

#### 3. \*\*核心功能对比\*\*（关键差异）：

特性	synchronized	ReentrantLock
**可中断等待**	□ 不支持（线程阻塞后无法响应 `interrupt()`）	□ `lockInterruptibly()` 支持中断
**超时获取**	□ 不支持	□ `tryLock(long time, TimeUnit unit)` 支持
**公平锁**	□ 仅非公平（JVM 保证）	□ 构造时传 `true` 可启用公平模式：`new ReentrantLock(true)`
**条件队列**	□ 仅 `wait()/notify()`（绑定单一条件）	□ `newCondition()` 可创建多个 `Condition`，精准唤醒
**锁信息监控**	□ 不可查询持有线程、等待队列等	□ 提供 `getHoldCount()`、`hasQueuedThreads()` 等诊断方法

#### 4. \*\*性能演进（JDK 6+ 重要事实）\*\*：

- JDK 6 引入 \*\*偏向锁 → 轻量级锁 → 重量级锁\*\* 升级机制，\*\*在无竞争或低竞争场景下

, `synchronized` 性能  $\approx$  甚至优于 `ReentrantLock`\*\*;

- `ReentrantLock` 是纯 Java 实现, 有方法调用开销, 但\*\*高竞争场景下因支持自旋、更细粒度调度, 吞吐量可能更高\*\* \* (补充知识点) \*。

5. \*\*可重入性与内存语义\*\*:

- 二者均 \*\*支持可重入\*\* (同一线程可重复获取同一锁);

- 均 \*\*保证 Happens-Before 关系\*\* , 禁止指令重排序, 提供内存可见性保障。

## □□ 易错点提醒

- □ “`synchronized` 效率一定比 `ReentrantLock` 低” → □ JDK 6+ 优化后, 低争用下 `synchronized` 更快, 且无手动释放风险;

- □ “`ReentrantLock` 默认是公平锁” → □ 默认\*\*非公平\*\* (构造函数无参即 `new ReentrantLock()`), 公平性需显式指定;

- □ “`synchronized` 不能实现可中断” → □ 严格说它\*\*不提供可中断的获取方式\*\* , 但线程中断状态仍可被检测 (只是锁本身不响应);

- □ “`unlock()` 可以放在 `catch` 块里” → □ 错! 必须放 `finally` , 否则异常时未释放会导致死锁。

□ 面试官可能追问:

- `synchronized` 的锁升级过程具体是怎么触发和转换的?

- `ReentrantLock` 的 AQS 队列中 Node 的 waitStatus 含义有哪些?

- 如果用 `synchronized` 实现一个带超时的资源获取, 怎么设计?

## □ 一句话总结

`synchronized` 简洁安全, 适合大多数场景; `ReentrantLock` 灵活强大, 只在需要可中断、超时、多条件等高级特性时选用。